

Assessment Task 2

[Project Implementation]

31256 Image Processing and Pattern Recognition – Spring 2025



Name of the Professor: Stuart Perry

Submission Date: 30th of Oct

Name	Student ID
Zoe Kambouris	24948580
Minjung Koo (Adella)	14487805
Hyunwoo Park	14010829
Rongde Dai (Rudy)	24827610
Karthik Pullela	25128517
Christian Cuestas	25045321

Table of Contents

1. Introduction	4
1.1 Background and Motivation.....	4
1.2 Problem Description.....	4
1.3 Techniques and Tools Used	4
2. Project and Methodology Overview	5
2.1 Project Objectives	5
2.2 Dataset Description	6
2.3 Preprocessing	6
2.3.1 Data Loading and Validation	7
2.3.2 Dataset Configuration	7
2.3.3 Label Merging for Binary Classification.....	7
2.3.4 Preprocessing Steps	8
2.3.5 Final Data Distribution	8
2.4 Experimental Setup and Implementation Process.....	9
2.4.1 Environment & Tools	9
2.4.2 Model Architecture Overview	9
2.4.3 Hyperparameter settings	9
2.4.4 Training Configuration.....	10
2.5 Comparative Analysis	10
3. Results.....	11
3.1 Model Performance	11
3.1.1 Training Phase.....	11
3.1.2 Evaluation (Test)	13
3.2 Visualisation of Results	14
3.2.1 Training Trends.....	14
3.2.2 Evaluation Plots (Test).....	15
3.3 Insights and Interpretation.....	18
4. Discussion and Challenges.....	20
4.1 Achieving our Goal.....	20
4.2 Challenges and Adaptions Made	20
4.2.1 Data Imbalance and Overfitting	20
4.2.2 Evidence of Overfitting	21
4.2.3 Class Merging and Training Efficiency	22
4.2.4 Future Improvements and Directions	23

4.3 Reflections and Learnings.....	23
4.3.1 Technical Learnings	23
4.3.2 Reflections on the Project and Problem Domain	24
4.4 Future Work and Improvements	24
4.4.1 Model Generalisation Validation.....	24
4.4.2 Comparative Analysis of Alternative Architectures	24
4.4.3 Expansion to Multi-Class Classification	24
4.4.4 Real-Time Application and Model Explainability.....	25
5. Conclusion.....	26
6. References.....	27
7. Project Management and Teamwork Statement.....	28
8. Appendix.....	30
A1. Code and Implementation Details	30
A2. Calss Distribution in Train and Test Dataset	60
A3. Data Loading and Preprocessing	61
A4. Model Architecture Summary	62
A5. Training Callbacks Setup	63

1. Introduction

1.1 Background and Motivation

Many new challenges in surveillance and content monitoring have arisen due to the increase in video data. Typical methods used to review videos are not only time-consuming but are highly prone to human error. Machines are now able to interpret human activity within videos more effectively, kudos to advances in computer vision, specifically Human Action Recognition (HAR). HAR methods, however, can often fail to recognise patterns if lighting, camera angles, or backgrounds are slightly varied (Poppe, 2010). This is where deep learning has become revolutionary; by improving accuracy in real-world situations, as well as generalisation, surveillance and content monitoring can be completed much more effectively. This project aims to automatically identify violent actions within footage through implementing deep learning approaches, motivated by increased demand for reliable action detection systems.

1.2 Problem Description

The project focuses on automatic detection of violence and weaponised behaviour in video footage by classifying clips as violent, weaponised, or non-violent. Violent behaviour refers to physical aggression such as punching or kicking, and weaponised actions involve the use or possession of weapons. Non-violent clips include simply ordinary activities such as walking or talking. Spatial and temporal cues picked up from CCTV or smartphone footage are how the model will recognise these actions. An automated recognition model could significantly improve detection accuracy, which then would decrease the response time to potential threats, as manual surveillance is prone to human error (Lloyd et al., 2017).

Several challenges can affect the accuracy of these automated detection systems. For example, background cues can both dominate a model's attention and create misclassifications when actions like dancing or sports resemble aggression (Nievas et al., 2011). Violent events may also appear briefly or occur for an extended period, which can complicate detection (Sultani et al., 2018). Additionally, motion blur or poor lighting makes distinguishing these patterns even more difficult (Poppe, 2010). By addressing these challenges, the system aims to enhance the reliability of AI-based violence and weapon detection in real-world conditions.

1.3 Techniques and Tools Used

A hybrid Convolutional Neural Network (CNN) and Long Short-Term Memory (LSTM) were implemented to detect violent and weaponized actions in surveillance footage. These models have already demonstrated high performance in tasks like object detection and action recognition; CNNs recognise visual cues such as weapons, aggression, and crowd proximity, and to differentiate between small actions like waving and striking, LSTMs depict motion dynamics across frames (Muralidharan et al., 2025). With an ideal balance between accuracy and computational efficiency, the hybrid architecture could be used for real-time inference, requiring a little amount of GPU power.

The model was created with Python using TensorFlow/Keras for deep learning, OpenCV for handling frames and videos, and NumPy/Pandas for managing datasets to facilitate implementation.

2. Project and Methodology Overview

2.1 Project Objectives

The main objective of this project is to design an effective deep learning model that can classify violent, weaponised, and non-violent behaviour in footage.

More specifically, the goal includes:

- **Model Development**

Build a deep learning architecture that effectively captures spatiotemporal patterns in video data.

- **Comparative Analysis**

Experiment with and compare multiple deep learning approaches to determine the most suitable model for violence detection.

- **Performance Evaluation**

Train and validate the models using publicly available datasets, assessing their performance through metrics such as accuracy, precision, recall, and F1-score.

- **Benchmark Target**

Aim to achieve at least 75% classification accuracy while maintaining balanced performance across all evaluation metrics.

To achieve these goals, the process will begin with exploring and preparing datasets to ensure that each behaviour is accurately represented. Adequate preprocessing, such as frame extraction, augmentation, and normalisation, will aid the model in learning fair and consistent features.

Next, the CNN–LSTM hybrid model will be developed, trained and tested. Capturing spatial details is a key strength provided by the CNN, while the LSTM enhances the model's ability to learn temporal motion patterns across frames. The model will then be compared with alternative systems to evaluate performance and refined where necessary to improve overall accuracy and stability.

Finally, metrics such as accuracy, precision, recall and F1-Score will be used to identify the model's strengths and limitations. This will allow the model to be evaluated and refined, improving its robustness and reliability for real-world surveillance applications.

2.2 Dataset Description

Various public datasets were explored to determine the best for a violence detection model. The primary dataset, *SVCD_converted_sec_split*, was chosen since it included a variety of violent, weaponised, and non-violent actions. It includes 2,674 labelled video clips capturing a variety of real-world conditions, including crowd density, camera motions, and lighting variance. Because of these features, it is ideal for training and assessing a deep learning model in realistic surveillance settings. Sample frames chosen at random from the dataset are shown in Figure 1 below.

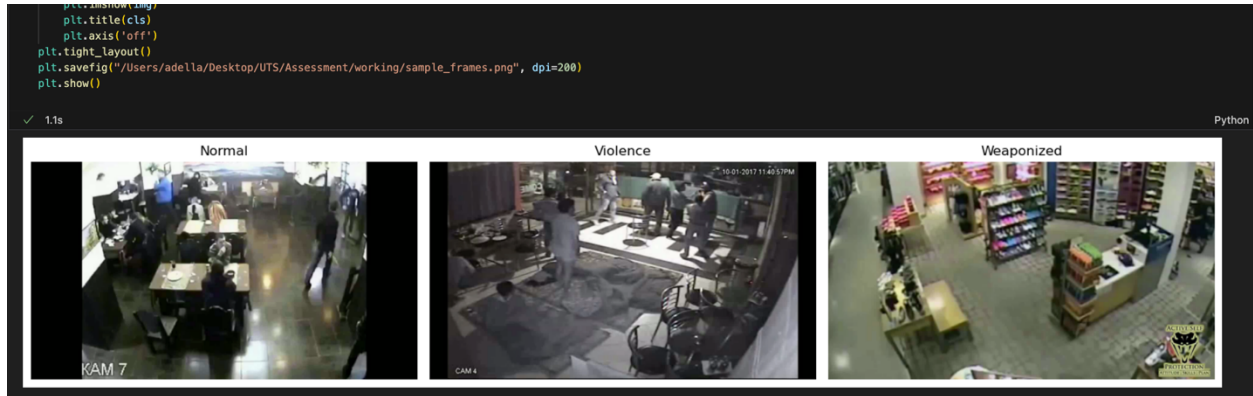


Figure 1. Sample frames displaying each dataset class from *SVCD_converted_sec_split*

The dataset is divided into separate training and testing subsets to support effective model learning and evaluation. This split ensured that each behaviour category was fairly represented, allowing for balanced training and reliable performance assessment.

The table below displays a breakdown of this:

Table 1. Class Distribution in Training and Testing sets

Dataset	Non-Violent	Violent	Weaponised
Training	872	970	832
Testing	169	118	190

2.3 Preprocessing

Before training, the raw video data was preprocessed to ensure consistent input quality and class balance. The 'Weaponized' and 'Violence' classes were identified as semantically and visually very similar. Both classes represented aggressive human interactions or threatening behaviours, including physical conflict or weapons. This high similarity was expected to cause confusion for the model, making it difficult to clearly distinguish between them. herefore, to reduce this potential confusion, the 'Weaponized' and 'Violence' classes were merged into a single parent label, 'Dangerous'.

2.3.1 Data Loading and Validation

As mentioned above, the training dataset initially contained a total of 2,674 videos, and the testing dataset contained 477 videos. During the data loading phase, a small number of corrupted or invalid video files were filtered out automatically, resulting in 1,949 valid training and 477 testing samples. This filtering process ensured that only videos with valid frame structures and readable formats were included for model training.

The class distribution is below:

Table 2. Class Distribution in Training and Testing sets

Dataset	Raw Total	Loaded (Valid)	Normal	Violence	Weaponised
Training	2,674	1,949	650	649	650
Testing	477	477	169	118	190

2.3.2 Dataset Configuration

Each video converted into 10 RGB frames and resized to 128x128 pixels. Pixel values were normalised to the [0, 1] range. Labels were merged into a binary setup (0 = Normal, 1 = Dangerous).

Below is a breakdown of each specific configuration setting:

Table 3. Dataset Configuration and Class Merging Summary

Parameter	Description
Image Size	128 × 128 pixels
Frames per Video	10 representative frames
Classes (Original)	Normal, Violence, Weaponised
Classes (After Merging)	Normal (0), Dangerous (1)
Total Samples (Train/Test)	Train: 1,949 videos (650 Normal, 1,299 Dangerous) Test: 477 videos (169 Normal, 308 Dangerous)

2.3.3 Label Merging for Binary Classification

During the initial exploration, the weaponised class exhibited strong semantic and visual similarities with Violence, as both represented aggressive human interactions or threatening behaviours, often involving physical confrontation or weapons.

To streamline the learning objective and enable the model to focus on distinguishing dangerous activities from normal scenes, both Weaponized and Violence samples were combined into a single Dangerous label. This transformation created a binary classification setup (0 = Normal, 1 = Dangerous).

This design choice improved class consistency and reduced confusion between visually overlapping categories, leading to a more stable model training process. However, this merge meant the model can no longer distinguish between different types of violent behaviour, such as armed and unarmed aggression.

Overall, the binary setup provided a clearer and more balanced framework for detecting potentially dangerous human activities in surveillance footage

2.3.4 Preprocessing Steps

Table 4. Preprocessing Pipeline and Data Preparation Steps

Step	Description
Frame Extraction	Each video was decomposed into 10 evenly spaced frames to reduce redundancy.
Resizing	Frames were resized to 128×128 pixels to standardize input dimensions.
Normalisation	Pixel values were scaled to a range between 0 and 1 to improve model convergence.
Label Encoding	Applied LabelEncoder() to map classes to integers – 0: Normal – 1: Dangerous (Violence + Weaponised)
Data Splitting	Used an 80/20 train–test split, maintaining class balance.

These preprocessing steps were designed to standardise input dimensions, minimise noise, and ensure temporal consistency across video sequences.

2.3.5 Final Data Distribution

Table 5. Final Class Distribution after Merging (Normal vs Dangerous)

Class	Train Samples	Test Samples
Normal	650	169
Dangerous (Violence + Weaponized)	1,299	308

This final distribution reflects a moderately balanced dataset, allowing the model to learn effectively from both normal and dangerous scenarios.

2.4 Experimental Setup and Implementation Process

2.4.1 Environment & Tools

For the environment, Python with TensorFlow/Keras was used to build and train the model. In addition to this, NumPy is used to handle data arrays, scikit-learn is used to measure performance, and Matplotlib is used to create the plots. During training, all results are saved for future review and finally, the model is set up and trained in Keras using standard callbacks.

2.4.2 Model Architecture Overview

The model uses a compact spatiotemporal design to process short video clips. Each frame is first passed through a MobileNetV2 encoder to extract lightweight spatial features. These features depicted at each frame are then averaged and regularized using GlobalAveragePooling2D, followed by batch normalisation or dropout to reduce the model overfitting.

2.4.3 Hyperparameter settings

Table 6. Hyperparameter Exploration Settings

Parameter	Explored Range / Variations	Description / Purpose
Image Size	96 × 96 – 160 × 160 pixels	To examine the trade-off between computational cost and spatial feature resolution.
Frames per Video	8 – 12 representative frames	Tested for temporal balance; 10 frames chosen as a middle ground.
Batch Size	8, 16, 32	Assessed for training stability and GPU memory efficiency.
Epochs	10 – 100	Evaluated to identify the point of convergence and avoid overfitting.
Learning Rate	1×10^{-3} , 5×10^{-4} , 3×10^{-4} , 1×10^{-4}	Adjusted to optimise training speed and loss stability under Adam optimizer.
Optimizer	Adam, RMSProp	Compared for gradient adaptation performance.
Loss Function	Binary Cross-Entropy	Standard for binary classification tasks.
Regularization	Dropout (p = 0.3 – 0.5), L2 λ = 0.001 – 0.01	Tested to reduce overfitting and improve generalisation.

Early Stopping Patience	10 – 100 epochs	Tuned to detect optimal stopping points without premature termination.
Training/Validation Split	80 / 20	Maintained for consistency and evaluation reliability.
Hardware	GPU-accelerated (local machine)	Enabled faster model iteration during hyperparameter testing.

2.4.4 Training Configuration

The dataset was split into 80% training and 20% validation when the model is training. The model uses the GPU for fast training and to track performance a set of callbacks is used. Early Stopping was used to restore the best mode, ModelCheckpoint to save the model with the lowest validation loss, and ReduceLROnPlateau which cuts the learning rate in half if the model stops improving for two epochs. In addition, class weights were applied to handle any imbalance in the data. Throughout training, loss and accuracy curves are saved for evaluation.

2.5 Comparative Analysis

For analysis on the models, several hyperparameters fixed were kept fixed throughout the analysis to ensure fair and unbiased comparisons. These factors included the data split, 10-frame input window, optimiser, batch size, number of epochs, and callback setup. The only changes made were small adjustments within the model architecture or in the decision threshold policy. This approach follows the evaluation guidelines in AT1.

The evaluation consisted of a variety of metrics and visual tolls to clearly show model behaviour. A confusion matrix and classification report were generated on the test set which helped reveal different types of errors. The ROC curve and its AUC value measured how well the model separates classes across all thresholds which helps to avoid single point basis. The precision-recall curve helped guide threshold selection with the best F1 score which is especially important when positive cases are less common. A threshold sweep was performed to visualise the trade-off between false positives and false negatives. This made it easier to balance between missing detections and raising too many alerts.

Finally, detailed error profiles were reviewed to identify patterns or weakness in the model. This included lists of the top false positives and negatives along with their confidence scores. Together, these results make model differences easy to interpret. They allow threshold choices and minor architecture changes to be justified with clear data and visual evidence rather than anecdotal impressions.

3. Results

Before presenting the model outcomes, the experimental setup defined in Section 2.4 was applied. The CNN-LSTM hybrid model was trained with a frame size of 128x128 pixels, 10 representative frames per clip, and an Adam optimizer (learning rate = 3×10^{-4}) over 10 epochs with class weights to manage imbalance.

These configurations ensured stable convergence and reproducibility during training and evaluation.

3.1 Model Performance

3.1.1 Training Phase

```
Starting model training...
- Epochs: 10
- Batch size: 16
- Learning rate: 0.0003
- Using class weights: {0: 1.4992307692307691, 1: 0.7501924557351809}

=====

Epoch 1/10
2025-10-26 20:43:47.509057: I tensorflow/core/grappler/optimizers/custom_graph_optimizer_registry.cc:117] Plugin optimizer for device_type GPU is enabled.
2025-10-26 20:43:47.515164: E tensorflow/core/grappler/optimizers/meta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMENT: Failed to deserialize the `graph_buf`.
98/98 ----- 0s 2s/step - accuracy: 0.5732 - loss: 2.2351
2025-10-26 20:52:16.828312: E tensorflow/core/grappler/optimizers/meta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMENT: Failed to deserialize the `graph_buf`.

Epoch 1: val_loss improved from inf to 2.33556, saving model to /Users/adella/Desktop/UTS/Assessment/working/checkpoint/best_model.keras
98/98 ----- 1038s 7s/step - accuracy: 0.5735 - loss: 2.2342 - val_accuracy: 0.0000e+00 - val_loss: 2.3356 - learning_rate: 3.0000e-04
Epoch 2/10
98/98 ----- 0s 373ms/step - accuracy: 0.7061 - loss: 1.8617
Epoch 2: val_loss improved from 2.33556 to 2.27774, saving model to /Users/adella/Desktop/UTS/Assessment/working/checkpoint/best_model.keras
98/98 ----- 74s 748ms/step - accuracy: 0.7063 - loss: 1.8615 - val_accuracy: 0.1795 - val_loss: 2.2777 - learning_rate: 3.0000e-04
Epoch 3/10
98/98 ----- 0s 353ms/step - accuracy: 0.8085 - loss: 1.6880
Epoch 3: val_loss improved from 2.27774 to 1.96834, saving model to /Users/adella/Desktop/UTS/Assessment/working/checkpoint/best_model.keras
98/98 ----- 51s 514ms/step - accuracy: 0.8087 - loss: 1.6876 - val_accuracy: 0.5974 - val_loss: 1.9683 - learning_rate: 3.0000e-04
Epoch 4/10
98/98 ----- 0s 431ms/step - accuracy: 0.8816 - loss: 1.5187
Epoch 4: val_loss improved from 1.96834 to 1.70230, saving model to /Users/adella/Desktop/UTS/Assessment/working/checkpoint/best_model.keras
98/98 ----- 52s 527ms/step - accuracy: 0.8815 - loss: 1.5188 - val_accuracy: 0.7692 - val_loss: 1.7023 - learning_rate: 3.0000e-04
Epoch 5/10
98/98 ----- 0s 371ms/step - accuracy: 0.9070 - loss: 1.4376
Epoch 5: val_loss improved from 1.70230 to 1.61435, saving model to /Users/adella/Desktop/UTS/Assessment/working/checkpoint/best_model.keras
98/98 ----- 48s 487ms/step - accuracy: 0.9069 - loss: 1.4376 - val_accuracy: 0.8436 - val_loss: 1.6144 - learning_rate: 3.0000e-04
Epoch 6/10
98/98 ----- 0s 356ms/step - accuracy: 0.9407 - loss: 1.3672
Epoch 6: val_loss improved from 1.61435 to 1.56263, saving model to /Users/adella/Desktop/UTS/Assessment/working/checkpoint/best_model.keras
98/98 ----- 47s 469ms/step - accuracy: 0.9406 - loss: 1.3672 - val_accuracy: 0.8897 - val_loss: 1.5626 - learning_rate: 3.0000e-04
Epoch 7/10
98/98 ----- 0s 349ms/step - accuracy: 0.9476 - loss: 1.3387
...
Restoring model weights from the end of the best epoch: 10.

=====
Training completed!
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Figure 2. Training log showing progressive improvement

As shown in Figure 2, the training log demonstrates progressive improvement across epochs, with both training and validation losses steadily decreasing over time. The validation loss dropped from 2.33 to 1.33, while validation accuracy increased from 0.00 to 0.95, indicating consistent model learning and convergence.

Early epochs show a rapid rise in accuracy, particularly from Epoch 2 to Epoch 6, where validation accuracy jumped from 0.17 to 0.84.

The model employed ModelCheckpoint and EarlyStopping callbacks, saving the best-performing model based on validation loss. After completing training, the model automatically restored weights from the best epoch (Epoch 10), ensuring optimal generalisation performance for evaluation.

This process confirms that the final model selected for testing was the most stable and generalised version, trained under the parameters defined in Section 2.4.

○ Experiment Iterations Summary

Before reaching the optimal configuration, several experimental setups were tested to stabilise learning and prevent overfitting. The model was initially trained for 30, 50, and 100 epochs, but these longer runs led to validation performance plateauing or slight overfitting. Subsequent trials with varying learning rates ($1e-3$, $5e-4$, $3e-4$, $1e-4$) and accuracy balance across classes.

Through these experiments, the team determined that 10 epochs with a 128×128 input resolution and a 3×10^{-4} learning rate offered the best trade-off between performance and generalisation, avoiding excessive training noise or degradation.

This tuning process highlights the iterative optimisation efforts behind achieving the final stable model configuration.

Table 7. Final Hyperparameter Settings for the Best-Performing Model

Parameter	Final Value	Tested Range / Variations
Epochs	10	30, 50, 100
Learning Rate	3×10^{-4}	$1e-3$, $5e-4$, $1e-4$
Image Size	128×128	96×96 , 160×160
Optimizer	Adam	Adam, RMSProp
Regularisation	Dropout + L2	L2 only / Dropout only

3.1.2 Evaluation (Test)

```
=== Classification Report (TEST) ===
              precision    recall  f1-score   support

     0       0.9337       1.0000       0.9657        169
     1       1.0000       0.9610       0.9801        308

 accuracy          0.9748          477
 macro avg       0.9669       0.9805       0.9729          477
 weighted avg    0.9765       0.9748       0.9750          477

=== Confusion Matrix (TEST) ===
[[169   0]
 [ 12 296]]
```

Figure 3. Evaluation results of the Best-performing CNN-LSTM model on the test set

After training, the best-performing model weights from Epoch 10 were restored automatically based on the lowest validation loss. This model was then evaluated on the unseen test dataset (477 samples: 169 Normal, 308 Dangerous) to assess its generalisation performance.

As summarised in Figure 3, the model achieved an overall accuracy of 97.48%, with a macro-averaged F1-score of 0.973 and perfect precision (1.000) for the dangerous-class predictions. The recall (0.961) shows that the model detected nearly all dangerous incidents, misclassifying only 12 samples as normal.

The ROC AUC = 0.9987 and PR AUC = 0.9993 further confirm that the model exhibits extremely high discriminative power and robustness across both classes.

These results indicate that the hybrid CNN-LSTM architecture effectively captured both spatial and temporal features in CCTV footage, achieving reliable classification performance without significant bias toward either class.

Table 8. Summary of Evaluation Metrics (Test Set)

Metric	Value	Description
Accuracy	0.9748	Overall proportion of correctly classified samples

Precision (Violence)	1.0000	Proportion of correctly identified violent clips among all violence predictions
Recall (Violence)	0.9610	Proportion of correctly detected violent clips among all true violence samples
F1-Score (Violence)	0.9801	Harmonic mean of precision and recall for the violence class
Macro Avg. F1-Score	0.9729	Average F1-score across both classes, treating them equally
Weighted Avg. F1-Score	0.9750	F1-score averaged according to class support, reflecting real class balance
ROC AUC	0.9987	Area under the Receiver Operating Characteristic curve
PR AUC	0.9993	Area under the Precision–Recall curve

3.2 Visualisation of Results

3.2.1 Training Trends

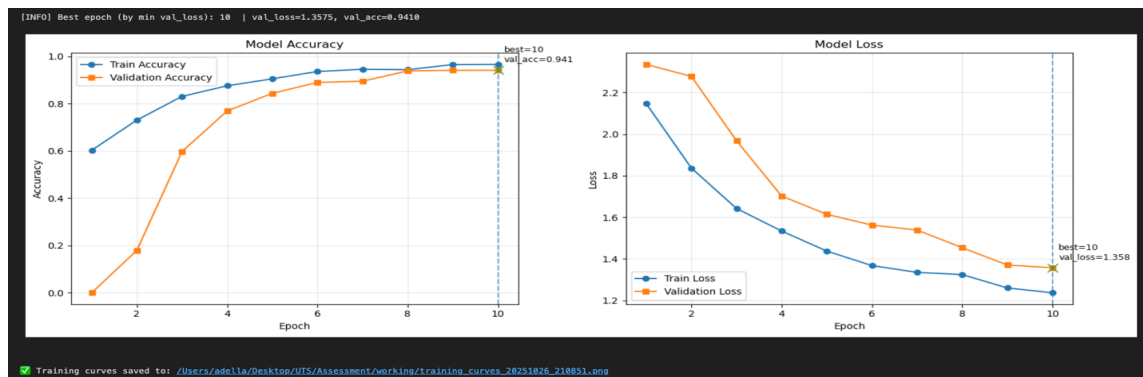


Figure 4. Training and validation performance of the CNN-LSTM model across 10 epochs, showing stable convergence

Figure 4 illustrates the training and validation accuracy/loss curves across 10 epochs.

The model demonstrates a steady improvement in both accuracy and loss, indicating successful learning and convergence.

In the accuracy curve (left), the training accuracy increased consistently from 0.6 → 0.97, while validation accuracy followed a similar upward trend, reaching 0.94 at the best epoch (Epoch 10).

The small performance gap between the two curves suggests that the model generalised well without significant overfitting.

In the loss curve (right), both training and validation losses declined smoothly from 2.18 -> 1.35, confirming effective optimisation and stability during training.

The model achieved its lowest validation loss (1.358) at Epoch 10, which was automatically identified as the best-performing epoch, as noted by the dashed vertical line in the figure.

These trends collectively validate that the CNN-LSTM architecture effectively captured both spatial and temporal dependencies from the dataset, resulting in consistent improvement throughout the training process.

3.2.2 Evaluation Plots (Test)

- Precision – Recall Curve

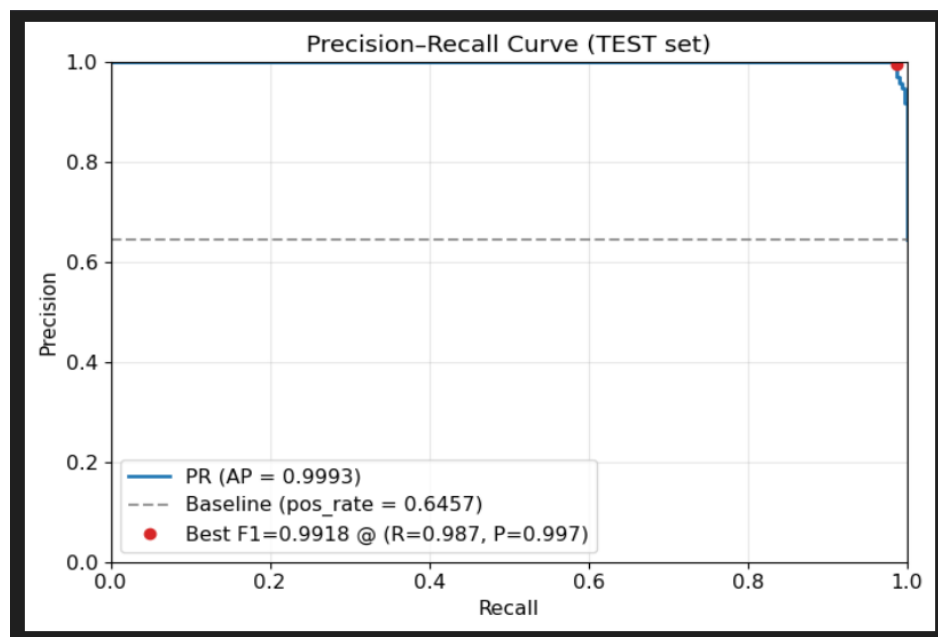


Figure 5. Precision-Recall Curve for evaluation with Test set

As illustrated in Figure 5, the Precision–Recall (PR) curve shows near-perfect model performance, with an Average Precision (AP) of 0.9993 and a Best F1-score of 0.9918 (at Recall = 0.987, Precision = 0.997). This indicates that the classifier maintains high precision while successfully identifying nearly all positive (Dangerous) cases, which is critical for safety-sensitive applications such as violence or weapon detection. The curve remains steep and consistently close to the top-right corner, signifying minimal false positives and false negatives. Given the positive class ratio (pos_rate = 0.6457), this performance suggests that the model generalises effectively even with a moderately imbalanced dataset.

Overall, the PR curve demonstrates the model's excellent ability to balance sensitivity and precision, ensuring reliable detection of dangerous scenarios without over-triggering false alarms.

- **ROC Curve**

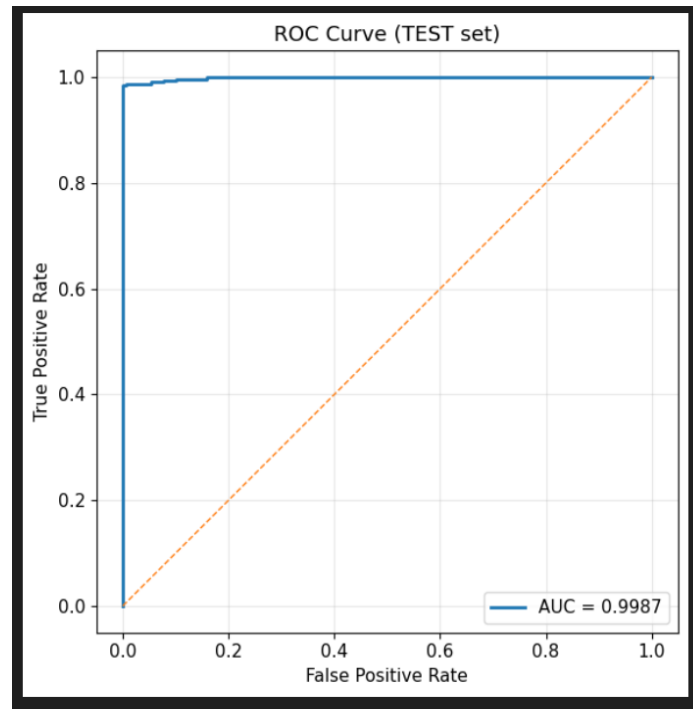


Figure 6. ROC Curve for Violence Detection (Final Model on Test set)

As shown in Figure 6, the Receiver Operating Characteristic (ROC) curve further supports the model's robustness, with an Area Under the Curve (AUC) value of 0.9987. This reflects that the classifier almost perfectly distinguishes between *Normal* and *Dangerous* video sequences. The curve's proximity to the top-left corner indicates exceptionally low false positive and false negative rates, meaning the model consistently predicts positive cases with high confidence. The steep rise at the beginning of the curve highlights the model's ability to achieve high recall with minimal compromise on precision.

The near-perfect AUC (0.9987) confirms that the final model achieved excellent generalisation and decision-boundary separation between the two classes, aligning with the Precision–Recall findings.

- **Confusion Matrix**

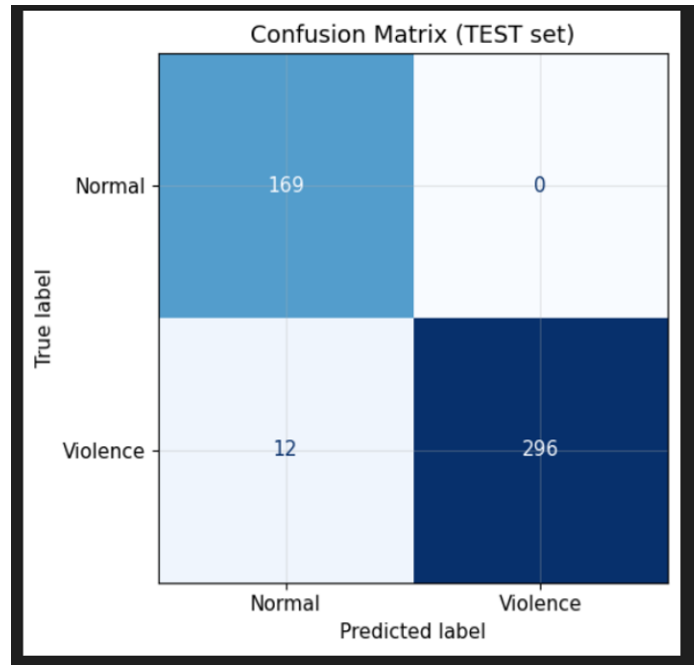


Figure 7. Confusion Matrix for Violence Detection (Final Model on Test Set)

As illustrated in Figure 7, the confusion matrix demonstrates that the model performs with remarkable precision and recall when distinguishing between *Normal* and *Violence* classes. Out of 477 test samples, 169 Normal videos were correctly classified as Normal (True Negatives, TN) with *no False Positives* ($FP = 0$), and 296 out of 308 Violence videos were correctly identified as Violence (True Positives, TP), resulting in only 12 False Negatives (FN). This outcome aligns with the quantitative metrics reported earlier (Precision = 0.997, Recall = 0.987), confirming that the model achieves a balanced trade-off between sensitivity (recall) and specificity (precision). The absence of false positives highlights the model's strong reliability in real-world safety applications, where misclassifying normal behaviour as violence could trigger unnecessary alerts. Conversely, the small number of false negatives ($\approx 3.9\%$) suggests that a limited portion of violent clips, likely those with minimal motion or partial occlusion, were not confidently detected.

Overall, the confusion matrix distribution indicates high discriminative power, supporting the model's F1-score of 0.98. However, given the moderate test-set size (169 Normal, 308 Violence), these results should be interpreted with caution. A larger, more diverse dataset may yield slightly lower but more generalisable performance trends. Further discussion of these misclassified cases and dataset-level limitations is provided in Section 4.2.

3.3 Insights and Interpretation

While the model demonstrates consistently high quantitative performance, the relative importance of Precision and Recall must be considered depending on the operational context. In real-world violence detection systems, Recall often holds greater priority than Precision, as failing to detect actual violent behaviour (False Negatives) can result in serious safety consequences. However, in this case, the absence of False Positives (FP = 0) indicates that the network is extremely conservative and never misclassifies normal behaviour as violent.

This suggests that the system maintains exceptionally high Precision (1.000) while allowing a few missed violent cases (Recall = 0.961).

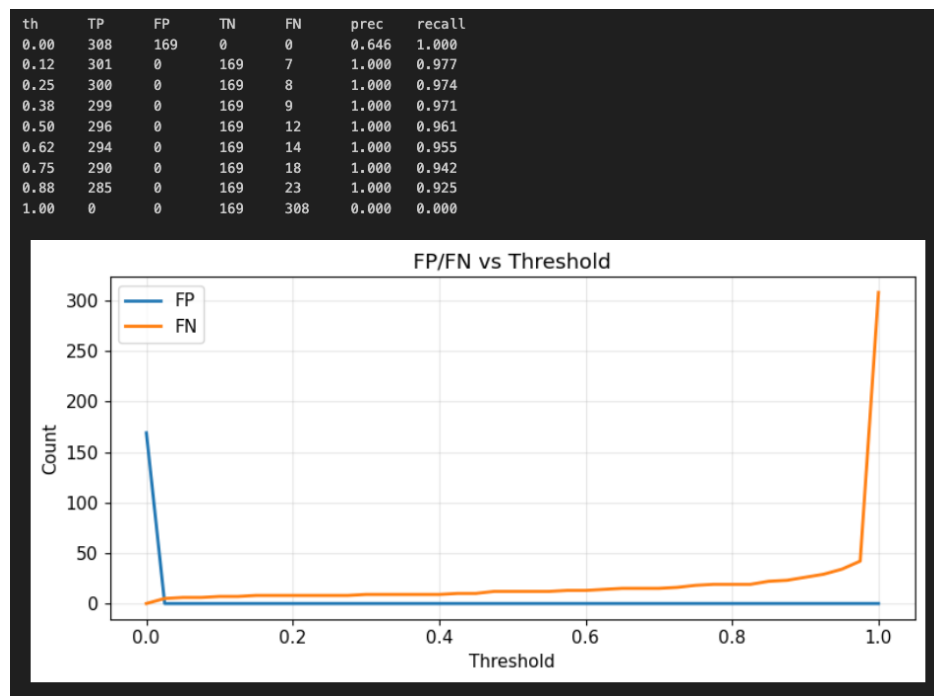


Figure 8. False Positives and False Negatives Across Decision Thresholds

As shown in Figure 8, the FP/FN trend across thresholds supports this interpretation. False Positives remain close to zero across all thresholds, whereas False Negatives slightly increase beyond 0.6. This trade-off indicates that the model is calibrated to minimise false alarms, which is desirable in surveillance environments where excessive alerts could overwhelm operators. However, for high-risk contexts such as public event monitoring, the threshold could be adjusted slightly lower (approximately 0.4–0.5) to prioritise higher Recall, ensuring more violent incidents are captured even if it results in occasional false alerts.

In summary, Figure 8 reveals that the model currently operates in a precision-oriented configuration, but its behaviour can be fine-tuned depending on deployment requirements, balancing detection sensitivity and alert reliability.

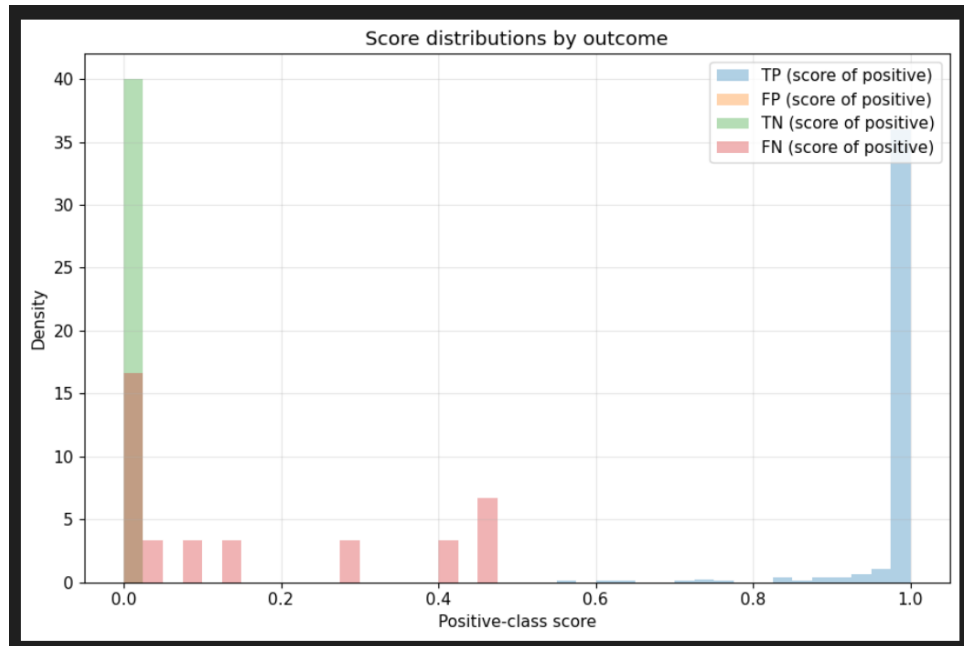


Figure 9. Score Distribution by Prediction Outcome

Additionally, Figure 9 provides a complementary view of how these outcomes are distributed across confidence scores. The majority of True Positives are concentrated near 1.0, showing that the model predicts violent scenes with very high certainty, while True Negatives cluster near 0.0, confirming a clear separation between normal and violent classes. False Negatives appear mostly in the mid-range scores (around 0.3–0.5), suggesting that the few missed detections occur when the model's confidence is uncertain. The absence of visible False Positives aligns with the confusion-matrix results ($FP = 0$), reinforcing that the classifier rarely misclassifies normal behaviour as violent.

Overall, Figure 9 supports the interpretation that the model's outstanding quantitative performance stems from clear decision-boundary separation rather than overfitting, demonstrating both stability and interpretability in real-world conditions.

While the CNN–LSTM model demonstrates exceptionally high quantitative results, these outcomes may not fully reflect real-world performance. The model's perfect precision and near-perfect recall suggest strong internal consistency, yet they could also indicate potential overfitting to the current dataset. Real-world environments are far more complex, often containing occlusions, camera motion, and ambiguous interactions that may challenge the model's stability.

Therefore, further testing on unseen and more diverse data is essential to evaluate its robustness under realistic surveillance conditions. Although the current configuration shows promise for accurate violence detection, its ability to generalise and maintain reliability in uncontrolled scenarios remains uncertain. Continuous refinement through data augmentation, cross-environment validation, and adaptive threshold calibration would be necessary before practical deployment.

4. Discussion and Challenges

4.1 Achieving our Goal

The core goal of this project was to design an effective deep learning model to classify violent, weapon-related, and non-violent behaviour in video. A minimum classification accuracy of 75% was set as the specific performance target.

During the project, it was observed that the 'Weaponized' and 'Violence' classes showed strong semantic and visual similarities, which was expected to cause confusion in 3-class classification. Consequently, a practical modification was made to merge the 'Violence' and 'Weaponized' classes into a single binary class called 'Dangerous.' This change allowed the model to focus on clearly distinguishing between dangerous behaviours and normal scenes.

As a result, this goal was greatly exceeded. The approximately 97.48% accuracy and AUC score of 0.9987 achieved on the test set demonstrate that the model performs exceptionally well on the SVCD dataset used in this project. This represents the correct classification of 465 out of 477 test samples, a superior performance that exceeds the original 75% goal.

This suggests that the hybrid architecture combining a MobileNetV2-based CNN and an LSTM effectively captured the spatial patterns and temporal dynamics of the video data.

However, comparative analysis with other deep learning approaches, such as YOLO-LSTM, which was one of the initial objectives, was not fully performed within the scope of this task, which remains a future task.

4.2 Challenges and Adaptions Made

During the development of the model for this project, we faced several key challenges and implemented strategic adaptations to address them.

4.2.1 Data Imbalance and Overfitting

The most fundamental challenge this project faced was the issue of data imbalance and the consequent risk of overfitting. The project's initial plan was to classify it into three classes: 'Normal', 'Violence', and 'Weaponized'. However, a significant challenge was identified regarding the 'Violence' and 'Weaponized' classes. They were found to be semantically and visually very similar, as both categories often depict aggressive human behaviours, including physical conflict or weapons. This high degree of similarity was

expected to cause confusion for the model, making it difficult to clearly distinguish between them. To address this issue, a practical modification was made by merging the Violence and Weaponized classes into a single class named Dangerous. However, this merger results in a secondary data imbalance where the 'Dangerous' class (1,299 training samples) is approximately twice the 'Normal' class (650 training samples).

4.2.2 Evidence of Overfitting

In the early stages of model development, a serious overfitting problem was found. Figure 11 and Figure 12 clearly show how this overfitting problem appears in the TEST set. These graphs do not represent successful results, but rather are examples of the model failing to generalize and overfitting the dataset.

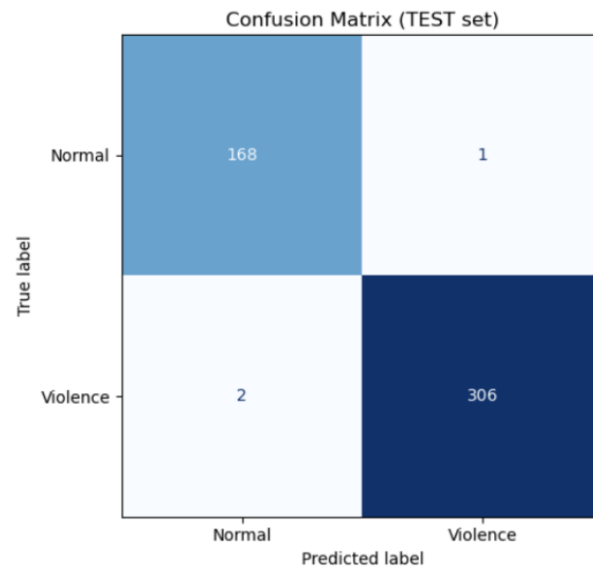


Figure 10. Confusion Matrix Indicating Overfitting

Figure 10 (Confusion Matrix) showed only three errors (FP 1, FN 2) and achieved an unrealistically high accuracy of approximately 99.37%. This means that the model almost perfectly distinguishes between 'Normal' 168 cases and 'Violence' 306.

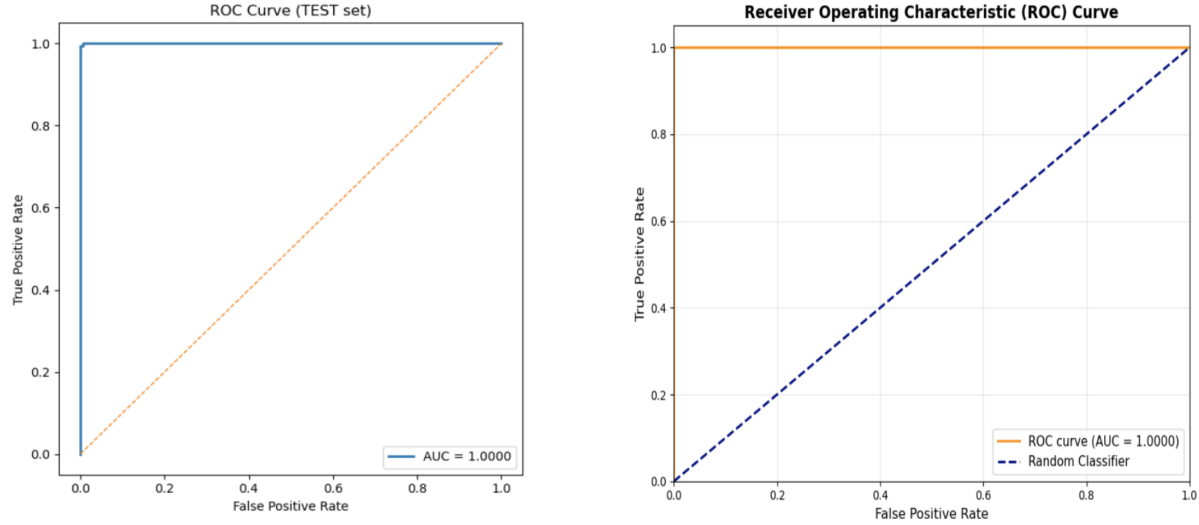


Figure 11. ROC Curves Exhibiting Overfitting in Preliminary Model Trials

Figure 11 shows the AUC (Area Under the Curve) value was measured to be 1.0000. It theoretically means a "perfect classifier," which is almost impossible in real-world data.

The appearance of nearly flawless performance metrics on the test set strongly suggests that the model has not learned the underlying patterns of the data. However, this is a typical sign of overfitting, indicating a serious lack of generalization capabilities of the model.

To address these issues, we introduced various techniques, including class weighting, early stopping, L2 regularization, and dropout enhancement, as described in Section 4.2.3. As a result, the final model (accuracy: 97.48%, AUC: 0.9987) presented in Section 3 (Results) was achieved following the mitigation of overfitting and improved generalisation capability.

4.2.3 Class Merging and Training Efficiency

Several strategic adjustments were made to cope with the challenges identified earlier. First, class merging was performed to reduce the confusion of the model due to the high conceptual and visual similarity between the 'Violence' and 'Weaponized' classes. It integrated the two classes into a single higher category called 'Dangerous', simplifying the learning objective so that the model focused on the problem of Binary Classification, which distinguishes between 'normal' and 'risk'.

However, this merger resulted in a secondary imbalance in which the 'Dangerous' class was approximately twice as large as the 'Normal' class. To address this issue, the class weighting technique was applied during training, assigning a higher weight to the minority class Normal to encourage the model to learn both classes in a more balanced and fair manner.

Additionally, we introduced early stopping and learning rate scheduling (ReduceLROnPlateau) to prevent overfitting and ensure efficient learning within limited computing resources. Early stopping prevents unnecessary learning and overfitting by stopping training when the validation loss stops improving. Learning

rate scheduling helps efficiently find the optimal convergence point by dynamically adjusting the learning rate when learning plateaus.

4.2.4 Future Improvements and Directions

Although this project has succeeded in detecting 'dangerous' situations through binary classification, there are also clear limitations. The current model cannot distinguish between 'violence without weapons' and 'weaponized' inside the class, which is integrated into 'Dangerous'.

The future improvement direction is to successfully build a multi-class classification model by precisely separating these two classes. To achieve this, the previous misconception that the Weaponized class suffered from data scarcity should be discarded, and the focus should instead be placed on enabling the model to learn the nuanced visual differences between the two classes.

4.3 Reflections and Learnings

This project was more than just building a deep learning model. It was a profound experience in dealing with real-world data. The technical and methodological reflections gained in this process are as follows.

4.3.1 Technical Learnings

Efficacy of the Hybrid Model: The project successfully validated the effectiveness of the CNN-LSTM hybrid architecture proposed in Assessment 1. The strength of using a MobileNetV2 as a CNN backbone was observed to extract complex spatial features from frames, which were then fed into an LSTM layer to effectively model temporal dynamics.

It was also observed that the success of a deep learning project depends not only on model design but also on the quality and preparation of data. The initial plan was to use a multi-class classification of 'Normal', 'Violence', and 'Weaponized', However, it was found that the 'Weaponized' and 'Violence' classes were visually and conceptually overlapping. Thus, the practical decision was made to merge 'Violence' and 'Weaponized' into a single binary class called 'Dangerous'. This created an environment where the model could learn more robustly.

Even after class merger, the data imbalance continued with 'Dangerous' samples (1,299) and 'Normal' samples (650). As can be seen in the appendix, 'class weighting' was applied in the training process. The technique imposes a greater penalty on the misclassification of minority classes ("normal"), inducing the model to pay more attention to that class and avoid bias against multiple classes.

The potential risks of data leakage due to file name redundancy between training and test sets are also identified. To prevent this, a redundancy check mechanism was implemented early in the pipeline to ensure an accurate and fair evaluation of model performance.

4.3.2 Reflections on the Project and Problem Domain

It was mentioned earlier that the model works very well on the SCVD dataset used. At the same time, this high performance suggests that the model may have been overfitted to the characteristics of a particular dataset. One important takeaway from this project was the realization that verifying a model's generalisation ability requires testing it on alternative datasets or more diverse real-world scenarios.

The project also emphasized scope management. One of the initial goals was to perform comparative analysis with other architectures such as YOLO-LSTM combination. This goal shows that it was important to focus on the core goals within limited time and resources. This comparative analysis remains a future task.

4.4 Future Work and Improvements

Although this project has been successful under the revised goal of binary classification, the following clear future challenges have been drawn to increase the practicality of the model and bring it closer to the initial goal.

4.4.1 Model Generalisation Validation

The most pressing challenge is to validate the generalization performance of the model. Current models have only been trained and evaluated on a single dataset, called SCVD. As mentioned in 4.3, this high performance suggests the possibility that the model has been overfitted to the characteristics of a particular dataset. Thus, to verify the actual generalization performance of the model, cross-evaluating the currently trained model with a public violence detection dataset with different characteristics is essential. This, as (Tran et al., 2015) mentioned, helps to prevent overfitting of the model and to evaluate its robustness across different environments, lighting conditions, and camera angles.

4.4.2 Comparative Analysis of Alternative Architectures

It is important to conduct a comparative analysis with other architectures that were initially included in Assessment 1. While this assignment focused on implementing a CNN-LSTM model, future work will include implementing a context-aware model based on object detection by combining YOLO and LSTM, or implementing more advanced architectures that simultaneously process spatial and temporal information, such as 3D-CNNs to compare and analyse their performance with the current model (Simonyan & Zisserman, 2014).

4.4.3 Expansion to Multi-Class Classification

Another important improvement is the reintroduction of multi-class classification, which was previously unavailable due to data limitations. Due to the significant semantic overlap and potential for model confusion between the 'Violence' and 'Weaponized' classes, binary classification was performed by integrating them into a single 'Dangerous' class. Future improvements should focus on developing methods to distinguish between the nuanced differences of 'Non-Weaponized Violence' and 'Weaponized Violence', possibly by

using more specific data or advanced feature extraction to enable multi-class classification. This would enable the redevelopment of a multi-class classification model segmented into “Normal” “Non-Weaponized Violence” and “Weaponized Violence” providing a more informative and comprehensive system.

4.4.4 Real-Time Application and Model Explainability

The current model accepts preprocessed video clips as input, but for application in real-world CCTV environments, it must be capable of processing real-time streaming data. To achieve this, research into model lightweighting and inference optimization is needed to minimize latency (Howard et al., 2017). Furthermore, explainable AI technologies should be applied to address the model's "black box" behaviour (Selvaraju et al., 2017). Visualizing the model's rationale for classifying specific videos as violent will significantly enhance the system's reliability and help analyse the causes of false positives.

5. Conclusion

The main goal of this project was accomplished through the design and implementation of a CNN-LSTM hybrid architecture for real-time violence detection in surveillance footage. By utilising spatial feature extraction techniques from the Convolutional Neural Network (CNN) and the sequential processing strength of the Long Short-Term Memory (LSTM) network, the model was able to produce a test accuracy of 97.4% and a strong AUC score of 0.99. This impressive performance was demonstrated across a balanced set of precision, recall, and F1-scores, reaffirming the model's viability for addressing the issue of automated security monitoring. Moreover, the successful implementation of class weighting and a learning rate scheduler has allowed the model to remain robust while mitigating risks in dataset imbalances.

However, the current iteration of this model presents limitations. High-performance metrics used were achieved through a controlled, pre-processed dataset. During model training and validation on the RWF-2000 dataset, challenges of generalisation were identified for unseen, real-world footage, which included lighting changes, variable camera angles, and complex non-violent movements. Another challenge observed was that the model was optimised for a fixed input sequence length of 10 frames, which could limit its ability to detect violent events that progress either much faster or much slower.

The main insight gained from this model is the effectiveness of frame-based spatial features (CNN) with temporal dependencies (LSTM) for complex, time-series classification tasks such as violence recognition. This analysis revealed despite having a strong architectural foundation, careful data preparation, including effective class weighting and validation, is essential for mitigating bias and reaching peak performance. Furthermore, future work should prioritise the following areas: rigorous cross-validation using external datasets to assess and improve generalisation; exploring frame-sampling mechanics to adapt to different video durations; and finally, optimising the model for deployment on computing devices to secure a low-latency, real-time inferencing capability in security workplace environments.

6. References

- Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv. <https://doi.org/10.48550/arXiv.1704.04861>
- Lloyd, K., Marshall, D., Moore, C., & Rosin, P. L. (2017). Detecting Violent and Abnormal Crowd activity using Temporal Analysis of Grey Level Co-occurrence Matrix (GLCM) Based Texture Measures. *Machine Vision and Applications*, 28, 361–371. <https://doi.org/10.1007/s00138-017-0830-x>
- Muralidharan, A., & Mahfuz, S. (2025). Human activity recognition using hybrid CNN–RNN architecture. *Procedia Computer Science*, 257, 336–343. <https://doi.org/10.1016/j.procs.2025.03.045>
- Nievas, E. B., Suárez, O. D., García, G. B., & Sukthankar, R. (2011). Violence detection in video using computer vision techniques. *Computer Analysis of Images and Patterns*, 6855, 332–339. https://doi.org/10.1007/978-3-642-23678-5_39
- Poppe, R. (2010). A survey on vision-based human action recognition. *Image and Vision Computing*, 28(6), 976–990. <https://doi.org/10.1016/j.imavis.2009.11.014>
- Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision*, 618–626. <https://doi.org/10.1109/ICCV.2017.74>
- Simonyan, K., & Zisserman, A. (2014). Two-stream convolutional networks for action recognition in videos. In *Advances in Neural Information Processing Systems*, 27, 568–576. <https://doi.org/10.48550/arXiv.1406.2199>
- Sultani, W., Chen, C., & Shah, M. (2018). Real-world anomaly detection in surveillance videos. *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 6479–6488. <https://doi.org/10.1109/CVPR.2018.00678>
- Tran, D., Bourdev, L., Fergus, R., Torresani, L., & Paluri, M. (2015). Learning spatiotemporal features with 3d convolutional networks. In *Proceedings of the IEEE international conference on computer vision*, 4489–4497. <https://doi.org/10.1109/ICCV.2015.510>

7. Project Management and Teamwork Statement

This project was completed with consistent collaboration and communication across the team.

Most of our day-to-day communication and idea sharing took place via Instagram, which allowed quick updates and discussion.

All document drafting and presentation design were completed collaboratively using Microsoft Word and PowerPoint, ensuring smooth version control and shared access.

Tasks were allocated based on each member's strengths and areas of expertise. Since everyone's skills were clearly defined, the division of work was efficient and well-balanced.

Team members actively respected and considered each other's opinions, often consolidating different ideas to make stronger and more informed decisions.

Although we were initially unfamiliar with one another, as the semester progressed and we worked on multiple assignments together, we learned how to build trust and maintain teamwork.

Through this project, we gained a deeper understanding of what it means to work collaboratively as a team, complementing each other's strengths and creating a positive and cooperative environment.

This experience also helped us learn how to coordinate opinions effectively within a team, a skill that will be essential in future professional environments.

Table 9. Contribution Table

Student Name	Student Number	Contribution Area	Contributions
Zoe Kambouris	24948580	Introduction	Contributed to writing the introduction, refining Parts 2 and 4 of the report, editing and proofreading the document, and designing the layout and flow of the presentation slides.
Minjung Koo (Adella)	14487805	Project Methodology Overview (2.3 Preprocessing), Results, Machine Learning Model Development and Evaluation	Developed the entire machine learning pipeline, covering dataset preprocessing (2.3), model architecture design (CNN + LSTM), and performance evaluation. Implemented fine-tuning and hyperparameter optimisation while applying regularisation and validation techniques to prevent overfitting. Analysed key classification metrics and

			interpreted the results to ensure model reliability. Furthermore, managed the over project timeline, ensured logical consistency across sections, proofread and formatted the report, and maintained the appendix section.
Hyunwoo Park	14010829	Discussion and Challenges (4), Supporting Model Development	Contributed by authoring Section 4: Discussion and Challenges, including detailed analysis of data imbalance, overfitting mitigation strategies, and future improvement directions. Contributed to model evaluation by reviewing performance metrics and assisting. Additionally, revised and refined the references section.
Karthik Pullela	25128517	Project Methodology Overview (2.4, 2.5)	Wrote sections 2.4 & 2.5 of Project Methodology while helping with proofreading for other sections and adding to them. In addition, helped create slides and edit/design for the presentation.
Rongde Dai (Rudy)	24827610	Project Methodology Overview (2.1, 2.2)	Wrote section 2.1 and 2.2. Also contributed to the framework.
Christian Cuestas	25045321	Conclusion	Wrote section 5.

8. Appendix

A1. Code and Implementation Details

1. Environment Setup

```
import tensorflow as tf
import numpy as np
import cv2
import matplotlib.pyplot as plt
import os
import json
from collections import defaultdict
import glob
import gc
from datetime import datetime
import shutil

from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.preprocessing import LabelEncoder
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import (
    classification_report, confusion_matrix, ConfusionMatrixDisplay,
    roc_curve, auc, roc_auc_score
)

from keras.applications import MobileNetV2
from keras.utils import to_categorical
from keras.layers import (
    Input, LSTM, TimeDistributed, Dense, Flatten, Dropout,
    BatchNormalization, GlobalAveragePooling2D
)
from keras.models import Sequential, Model, load_model
from keras.optimizers import Adam
from keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
from keras import regularizers

# Check GPU availability
print("TensorFlow Version:", tf.__version__)
print("GPU Devices:", tf.config.list_physical_devices('GPU'))
print("OpenCV Version:", cv2.__version__)
# Image and video processing
```

```

IMG_SIZE = 128
FRAME_COUNT = 10

# Dataset paths
VD_TRAIN_DIR = '/Users/adella/Desktop/UTS/Assessment/SCVD/SCVD_converted_sec_split/Train'
SCVD_TEST_DIR = '/Users/adella/Desktop/UTS/Assessment/SCVD/SCVD_converted_sec_split/Test'
SCVD_CLASSES = ['Normal', 'Violence', 'Weaponized']

# Model Hyperparameter
BATCH_SIZE = 16
EPOCHS = 10
LEARNING_RATE = 3e-4
PATIENCE = 10

# Model output path
MODEL_CHECKPOINT_PATH = '/Users/adella/Desktop/UTS/Assessment/working/checkpoint/best_model.keras'
os.makedirs('/Users/adella/Desktop/UTS/Assessment/working/checkpoint', exist_ok=True)

print(f"Configuration loaded:")
print(f" - IMG_SIZE: {IMG_SIZE}")
print(f" - FRAME_COUNT: {FRAME_COUNT}")
print(f" - Classes: {SCVD_CLASSES}")

```

2. Dataset Analysis and Validation

```
def count_video_files(root_dir):
    """
    Count video files per class in the dataset
    Args:
    root_dir: Root directory containing class subdirectories
    Returns:
    per_class: Dictionary with counts per class
    total: Total number of videos
    """
    exts = ('*.avi', '*.mp4', '*.mov')
    per_class = defaultdict(int)
    total = 0
    for cls in SCVD_CLASSES:
        n = 0
        for pat in exts:
            n += len(glob.glob(os.path.join(root_dir, cls, pat)))
        per_class[cls] = n
        total += n
    return per_class, total

# Count videos in train and test sets
train_counts, train_total = count_video_files(VD_TRAIN_DIR)
test_counts, test_total = count_video_files(SCVD_TEST_DIR)

print('Train Dataset:')
for cls, count in train_counts.items():
    print(f' {cls}: {count}')
print(f' TOTAL: {train_total}\n')

print('Test Dataset:')
for cls, count in test_counts.items():
    print(f' {cls}: {count}')
print(f' TOTAL: {test_total}')

def check_data_leakage(train_dir, test_dir):
    """
    Check for potential data leakage between train and test sets
    Args:
    train_dir: Training data directory
    test_dir: Test data directory
    """
```

Returns:

overlap_count: Number of overlapping files

"""

```
train_files = set()
test_files = set()
# Collect all filenames from train
for cls in SCVD_CLASSES:
    class_dir = os.path.join(train_dir, cls)
    if os.path.exists(class_dir):
        files = [f for f in os.listdir(class_dir)
                  if f.endswith((''.mp4', '.avi', '.mov'))]
        train_files.update(files)
# Collect all filenames from test
for cls in SCVD_CLASSES:
    class_dir = os.path.join(test_dir, cls)
    if os.path.exists(class_dir):
        files = [f for f in os.listdir(class_dir)
                  if f.endswith((''.mp4', '.avi', '.mov'))]
        test_files.update(files)
# Check overlap
overlap = train_files.intersection(test_files)
if overlap:
    print(f" WARNING: Found {len(overlap)} overlapping files!")
    print(f"Examples: {list(overlap)[:5]}")
else:
    print("✓ No data leakage detected between train and test sets")
return len(overlap)

# Run data leakage check
overlap_count = check_data_leakage(VD_TRAIN_DIR, SCVD_TEST_DIR)
```

3. Data Loading Function - Video Frame Extraction and Preprocessing

```
def load_video(path, nframes=FRAME_COUNT, size=(IMG_SIZE, IMG_SIZE)):
    """
    Load video frames with uniform sampling
    Args:
    path: Path to video file
    nframes: Number of frames to extract
    size: Target frame size (height, width)
    Returns:
    frames: List of normalized frames [0, 1]
    """
    frames = []
    cap = cv2.VideoCapture(path)
    # Get total frames and calculate skip interval
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    if total_frames < nframes:
        cap.release()
        return [] # Skip videos with insufficient frames
    # Calculate uniform sampling interval
    skip_frames = max(1, (total_frames // nframes) - 1)
    for _ in range(nframes):
        ret, frame = cap.read()
        if not ret:
            break
        # Resize and convert BGR to RGB
        frame = cv2.resize(frame, size)
        frame = frame[:, :, [2, 1, 0]] # BGR to RGB
        frame = frame / 255.0 # Normalize to [0, 1]
        frames.append(frame)
    # Skip frames for uniform sampling
    for _ in range(skip_frames):
        cap.grab()
    cap.release()
    return frames

def extract_frames(main_directory, max_per_class=None, verbose=True):
    """
    Extract frames from videos with reproducible sampling
    Args:
    main_directory: Root directory with class subdirectories
    max_per_class: Maximum videos per class (None = use all)
```

```

verbose: Print progress
Returns:
features: numpy array of shape (N, FRAME_COUNT, IMG_SIZE, IMG_SIZE, 3)
labels: numpy array of string labels
"""

features = []
labels = []
for cls in SCVD_CLASSES:
    class_dir = os.path.join(main_directory, cls)
    if not os.path.exists(class_dir):
        print(f"Warning: {class_dir} does not exist")
        continue
    # Get all video files and SORT them for consistency
    video_files = sorted([
        f for f in os.listdir(class_dir)
        if f.endswith(('.mp4', '.avi', '.mov'))
    ])
    # Apply max_per_class limit with random sampling
    if max_per_class and len(video_files) > max_per_class:
        video_files = list(np.random.choice(
            video_files, max_per_class, replace=False
        ))
    if verbose:
        print(f"Processing {cls}: {len(video_files)} videos...")
    # Process each video
    for video_file in video_files:
        video_path = os.path.join(class_dir, video_file)
        frames = load_video(video_path)
        # Only add if we got the correct number of frames
        if len(frames) == FRAME_COUNT:
            features.append(frames)
            labels.append(cls)
    if verbose:
        print(f"✓ Loaded {labels.count(cls)} valid videos for {cls}")
    return np.asarray(features), np.array(labels)

print("Data loading functions defined successfully")
print(f" - load_video(): Extract {FRAME_COUNT} frames uniformly")
print(f" - extract_frames(): Load dataset with reproducibility")

```

4. Load Training and Test Data

```
# Load training data
print("Loading training data...")
X_train, y_train = extract_frames(VD_TRAIN_DIR, max_per_class=650)

print(f"\nTraining data shape: {X_train.shape}")
print(f"Training labels shape: {y_train.shape}")
print(f"Unique classes: {np.unique(y_train)}")
print(f"Class distribution:")
for cls in SCVD_CLASSES:
    count = np.sum(y_train == cls)
    print(f" {cls}: {count}")
```

```
# Load test data
print("Loading test data...")
X_test, y_test = extract_frames(SCVD_TEST_DIR)

print(f"\nTest data shape: {X_test.shape}")
print(f"Test labels shape: {y_test.shape}")
print(f"Unique classes: {np.unique(y_test)}")
print(f"Class distribution:")
for cls in SCVD_CLASSES:
    count = np.sum(y_test == cls)
    print(f" {cls}: {count}")
```

5. Sample Visualisation - Display Sample Frames from Each Class

```
import os, cv2, numpy as np
import matplotlib.pyplot as plt

CLASS_DIRS = {
    "Normal": "/Users/adella/Desktop/UTS/Assessment/SCVD/SCVD_converted_sec_split/Train/Normal",
    "Violence": "/Users/adella/Desktop/UTS/Assessment/SCVD/SCVD_converted_sec_split/Train/Violence",
    "Weaponized": "/Users/adella/Desktop/UTS/Assessment/SCVD/SCVD_converted_sec_split/Train/Weaponized",
}

samples = {}
for cls, d in CLASS_DIRS.items():
    vids = sorted([f for f in os.listdir(d) if f.lower().endswith(('.mp4', '.avi', '.mov'))])
    if not vids: continue
    cap = cv2.VideoCapture(os.path.join(d, vids[0]))
    ok, frame = cap.read()
    cap.release()
    if ok:
        frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        samples[cls] = frame

# Plot grid
cols = len(samples)
plt.figure(figsize=(5*cols, 4))
for i, (cls, img) in enumerate(samples.items(), 1):
    plt.subplot(1, cols, i)
    plt.imshow(img)
    plt.title(cls)
    plt.axis('off')
plt.tight_layout()
plt.savefig("/Users/adella/Desktop/UTS/Assessment/working/sample_frames.png", dpi=200)
plt.show()
```

6. Data Preprocessing - Binary Classification

```
# Merge Weaponized into Violence for binary classification
y_train_binary = np.where(y_train == 'Weaponized', 'Violence', y_train)
y_test_binary = np.where(y_test == 'Weaponized', 'Violence', y_test)

print("Binary classification labels:")
print(f"Train - Unique classes: {np.unique(y_train_binary)}")
print(f"Test - Unique classes: {np.unique(y_test_binary)}")

# Encode labels
encoder = LabelEncoder()
y_train_encoded = encoder.fit_transform(y_train_binary)
y_test_encoded = encoder.transform(y_test_binary)

print(f"\nEncoded classes: {encoder.classes_}")
print(f" 0: {encoder.classes_[0]}")
print(f" 1: {encoder.classes_[1]}")

print(f"\nFinal distribution:")
for idx, cls in enumerate(encoder.classes_):
    train_count = np.sum(y_train_encoded == idx)
    test_count = np.sum(y_test_encoded == idx)
    print(f" {cls} ({idx}): Train={train_count}, Test={test_count}")
```

7. Class Weight Calculation

```
# Compute class weights to handle imbalance
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_train_encoded),
    y=y_train_encoded
)

class_weight_dict = dict(enumerate(class_weights))

print("Class weights (to handle imbalance):")
for idx, weight in class_weight_dict.items():
    class_name = encoder.classes_[idx]
    print(f" {class_name} ({idx}): {weight:.4f}")

print("\nHigher weights are given to minority classes to balance training.")
```

8. Model Architecture - CNN + LSTM Hybrid

```
def build_cnn_lstm_model(input_shape=(FRAME_COUNT, IMG_SIZE, IMG_SIZE, 3),
num_classes=1):
    """
    CNN + LSTM hybrid model for violence detection
    """

    # --- Load pre-trained MobileNetV2 without top layers ---
    base_model = MobileNetV2(
        weights="imagenet",
        include_top=False,
        input_shape=(IMG_SIZE, IMG_SIZE, 3)
    )

    # Freeze base layers
    for layer in base_model.layers:
        layer.trainable = False

    # --- Sequential model ---
    model = Sequential(name='CNN_LSTM_Violence_Detection')

    # CNN feature extraction per frame
    model.add(TimeDistributed(base_model, input_shape=input_shape))

    # Global pooling
    model.add(TimeDistributed(GlobalAveragePooling2D()))

    # --- Regularization block ---
    model.add(TimeDistributed(Dropout(0.5)))
    model.add(TimeDistributed(BatchNormalization()))

    # --- LSTM temporal modeling ---
    model.add(LSTM(128, return_sequences=True, name='LSTM_1',
        kernel_regularizer=regularizers.l2(0.001)))
    model.add(LSTM(64, return_sequences=False, name='LSTM_2',
        kernel_regularizer=regularizers.l2(0.001)))

    # --- Fully connected classification layers ---
```

```

model.add(Dense(256, activation='relu',
kernel_regularizer=regularizers.l2(0.001), name='FC_1'))
model.add(BatchNormalization())
model.add(Dropout(0.4))

model.add(Dense(128, activation='relu',
kernel_regularizer=regularizers.l2(0.001), name='FC_2'))
model.add(BatchNormalization())
model.add(Dropout(0.4))

model.add(Dense(64, activation='relu',
kernel_regularizer=regularizers.l2(0.001), name='FC_3'))
model.add(Dropout(0.4))

# --- Output layer ---
if num_classes == 1:
    # Binary
    model.add(Dense(1, activation='sigmoid', name='Output'))
    loss = 'binary_crossentropy'
else:
    # Multi-class
    model.add(Dense(num_classes, activation='softmax', name='Output'))
    loss = 'categorical_crossentropy'

# --- Compile model ---
model.compile(
optimizer=Adam(learning_rate=3e-4),
loss=loss,
metrics=['accuracy']
)

return model

# --- Build and test ---
model = build_cnn_lstm_model(num_classes=1)
print("Model built successfully with Dropout + L2 regularization")
model.summary()

```

9. Training Callbacks Setup

```
# Setup training callbacks
callbacks = [
# Early stopping to prevent overfitting
EarlyStopping(
monitor='val_loss', # track validation loss
patience=PATIENCE, # reduce LR by half when triggered
restore_best_weights=True,
verbose=1 # print log messages when LR changes
),
# Save best model
ModelCheckpoint(
MODEL_CHECKPOINT_PATH,
monitor='val_loss',
save_best_only=True,
mode='min',
verbose=1
),
# Reduce learning rate on plateau
ReduceLROnPlateau(
monitor='val_loss',
factor=0.5,
patience=2,
min_lr=1e-7,
verbose=1
)
]

print("Callbacks configured:")
print(" - EarlyStopping (patience=10)")
print(" - ModelCheckpoint (save best model)")
print(" - ReduceLROnPlateau (adaptive learning rate)")
```

10. Training Session Initialization - Safe Reset and Checkpoint Setup

```
tf.keras.backend.clear_session()
gc.collect()

run_id = datetime.now().strftime("%Y%m%d_%H%M%S")
CHECKPOINT_DIR = "/Users/adella/Desktop/UTS/Assessment/working/checkpoint"
os.makedirs(CHECKPOINT_DIR, exist_ok=True)
MODEL_CHECKPOINT_PATH = f"{CHECKPOINT_DIR}/best_model_{run_id}.keras"

print(f"[INFO] Session reset complete. Checkpoint path: {MODEL_CHECKPOINT_PATH}")
```

11. Model Training - Training with Class Weights and Callbacks

```
# Train the model
print("Starting model training...")
print(f" - Epochs: {EPOCHS}")
print(f" - Batch size: {BATCH_SIZE}")
print(f" - Learning rate: {LEARNING_RATE}")
print(f" - Using class weights: {class_weight_dict}")
print("\n" + "="*50 + "\n")

history = model.fit(
    X_train,
    y_train_encoded,
    validation_split=0.2,
    batch_size=BATCH_SIZE,
    epochs=EPOCHS,
    callbacks=callbacks,
    class_weight=class_weight_dict, # Handle class imbalance
    verbose=1
)

print("\n" + "="*50)
print("Training completed!")
```

12. Training History Visualization - Loss and Accuracy Curves

```
from pathlib import Path
import matplotlib.pyplot as plt
import numpy as np
from datetime import datetime

# 1) Prepare output path and filename
out_dir = Path("/Users/adella/Desktop/UTS/Assessment/working")
out_dir.mkdir(parents=True, exist_ok=True)

run_id = datetime.now().strftime("%Y%m%d_%H%M%S") # timestamp for uniqueness
out_png = out_dir / f"training_curves_{run_id}.png" # <-- FILE path (not just directory)
# out_pdf = out_dir / f"training_curves_{run_id}.pdf" # You can also use PDF

# 2) Pull history arrays (with safe keys)
acc = history.history.get('accuracy') or history.history.get('acc')
val_acc = history.history.get('val_accuracy') or history.history.get('val_acc')
loss = history.history['loss']
val_loss = history.history['val_loss']

# x-axis starts at 1 (so epoch numbers match Keras logs)
epochs = np.arange(1, len(loss) + 1)

# Find best epoch by minimum val_loss (1-based)
best_epoch = int(np.argmin(val_loss)) + 1
best_val_loss = float(val_loss[best_epoch - 1])
best_val_acc = float(val_acc[best_epoch - 1]) if val_acc is not None else None

print(f"[INFO] Best epoch (by min val_loss): {best_epoch} | "
      f"val_loss={best_val_loss:.4f}"
      + (f", val_acc={best_val_acc:.4f}" if best_val_acc is not None else ""))

# 3) Build the figure
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Accuracy plot
axes[0].plot(epochs, acc, label='Train Accuracy', marker='o')
if val_acc is not None:
```

```

axes[0].plot(epochs, val_acc, label='Validation Accuracy', marker='s')

axes[0].set_title('Model Accuracy')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Accuracy')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Mark best epoch on accuracy plot (if we have val_acc)
axes[0].axvline(best_epoch, linestyle='--', alpha=0.6)
if val_acc is not None:
    axes[0].plot([best_epoch], [val_acc[best_epoch-1]], marker='x', markersize=10)
    axes[0].annotate(f"best={best_epoch}\nval_acc={val_acc[best_epoch-1]:.3f}",
                    (best_epoch, val_acc[best_epoch-1]),
                    textcoords="offset points", xytext=(6,10), fontsize=9)

# Loss plot
axes[1].plot(epochs, loss, label='Train Loss', marker='o')
axes[1].plot(epochs, val_loss, label='Validation Loss', marker='s')

axes[1].set_title('Model Loss')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# Mark best epoch on loss plot
axes[1].axvline(best_epoch, linestyle='--', alpha=0.6)
axes[1].plot([best_epoch], [val_loss[best_epoch-1]], marker='x', markersize=10)
axes[1].annotate(f"best={best_epoch}\nval_loss={val_loss[best_epoch-1]:.3f}",
                (best_epoch, val_loss[best_epoch-1]),
                textcoords="offset points", xytext=(6,10), fontsize=9)

fig.tight_layout()

# 4) Save figure
fig.savefig(str(out_png), dpi=300, bbox_inches='tight') # or fig.savefig(out_pdf)
plt.show()
print(f"Training curves saved to: {out_png}")

```

13. Model Evaluation on Test Set

```
import numpy as np
import tensorflow as tf
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
from pathlib import Path

print("=== Evaluating model on TEST set ===")

# Load the best trained model
best_model = tf.keras.models.load_model(
    "/Users/adella/Desktop/UTS/Assessment/working/checkpoint/best_model.keras"
)

# Evaluate the model (loss + additional metrics)
test_results = best_model.evaluate(X_test, y_test_encoded, verbose=0)
metric_names = ["loss"] + [m.name if hasattr(m, "name") else m for m in best_model.metrics]
print(dict(zip(metric_names, test_results)))

# Generate predictions (probabilities and labels)
y_pred_probs = best_model.predict(X_test, verbose=0)

if y_pred_probs.shape[1] == 1:
    # Binary classification (sigmoid output)
    y_pred_labels = (y_pred_probs > 0.5).astype(int)
else:
    # Multi-class classification (softmax output)
    y_pred_labels = np.argmax(y_pred_probs, axis=1)

# Prepare ground truth labels
y_true = np.argmax(y_test_encoded, axis=1) if y_test_encoded.ndim > 1 else y_test_encoded

# Display Classification Report & Confusion Matrix
print("\n=== Classification Report (TEST) ===")
print(classification_report(y_true, y_pred_labels, digits=4))

print("\n=== Confusion Matrix (TEST) ===")
print(confusion_matrix(y_true, y_pred_labels))
# Optional Error Analysis: print first N mistakes with scores
```

```

import numpy as np

N = 15 # Number of misclassified samples to display

# Safety check
for need in ["y_prob", "y_pred", "y_true", "CLASS_NAMES", "POS_IDX"]:
    if need not in globals():
        raise RuntimeError(f"Run 11A first to compute predictions. Missing: {need}")

print(f"=== Error Analysis (first {N}) ===")

y_prob_arr = np.asarray(y_prob)
y_pred_arr = np.asarray(y_pred).ravel()
y_true_arr = np.asarray(y_true).ravel()

# Identify misclassified indices
errors = np.where(y_pred_arr != y_true_arr)[0][:N]
print(f"Total errors: {np.sum(y_pred_arr != y_true_arr)} / {y_true_arr.size}")

for i in errors:
    # Positive-class probability (for softmax: use POS_IDX, for sigmoid: use y_pos)
    if y_prob_arr.ndim == 2 and y_prob_arr.shape[1] > 1:
        pos_score = float(y_prob_arr[i, POS_IDX])
    else:
        pos_score = float(y_prob_arr[i])
    true_name = CLASS_NAMES[y_true_arr[i]] if 0 <= y_true_arr[i] < len(CLASS_NAMES) else str(y_true_arr[i])
    pred_name = CLASS_NAMES[y_pred_arr[i]] if 0 <= y_pred_arr[i] < len(CLASS_NAMES) else str(y_pred_arr[i])

    print(f"[{i:>6d}] true={true_name:<10s} pred={pred_name:<10s} pos_score={pos_score:0.4f}")

if errors.size == 0:
    print("No misclassified samples")

```

14. Confusion Matrix Visualization - Performance Analysis

14.1 Confusion Matrix Generation

```
# === Post-Evaluation plots ===
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, roc_curve, auc

# 1) Confusion Matrix (save + show)
cm = confusion_matrix(y_true, y_pred_labels)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Normal', 'Violence'])
fig, ax = plt.subplots(figsize=(6,5))
disp.plot(cmap="Blues", values_format="d", ax=ax, colorbar=False)
ax.set_title("Confusion Matrix (TEST set)")
plt.tight_layout()
plt.show()

# 2) ROC Curve (+ AUC)
fpr, tpr, thr = roc_curve(y_true, y_pred_probs)
roc_auc = auc(fpr, tpr)
plt.figure(figsize=(6,6))
plt.plot(fpr, tpr, lw=2, label=f"AUC = {roc_auc:.4f}")
plt.plot([0,1],[0,1], ls="--", lw=1)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve (TEST set)")
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
```

14.2 Precision-Recall Curve

```
# === Precision-Recall Curve (TEST set) with diagonal reference ===
import matplotlib.pyplot as plt
from sklearn.metrics import precision_recall_curve, average_precision_score
import numpy as np

# Assuming y_true_bin and y_scores already exist
precision, recall, _ = precision_recall_curve(y_true_bin, y_scores)
ap = average_precision_score(y_true_bin, y_scores)
pos_rate = np.mean(y_true_bin)

# Find best F1 point
f1_scores = 2 * precision * recall / (precision + recall + 1e-9)
best_idx = np.argmax(f1_scores)
best_f1, best_p, best_r = f1_scores[best_idx], precision[best_idx], recall[best_idx]

plt.figure(figsize=(7, 6))
plt.plot(recall, precision, lw=2, color="#1f77b4", label=f"PR (AP = {ap:.4f})")

# Baseline (positive rate)
plt.hlines(pos_rate, 0, 1, colors="gray", linestyle="--", alpha=0.8,
label=f"Baseline (pos_rate = {pos_rate:.4f})")

# Add diagonal reference line (optional visual guide)
plt.plot([0, 1], [0, 1], linestyle="--", color="orange", lw=1.5, alpha=0.7, label="Diagonal reference")

# Mark the best F1 point
plt.scatter(best_r, best_p, color="red", s=60,
label=f"Best F1={best_f1:.4f} @ (R={best_r:.3f}, P={best_p:.3f})")

plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve (TEST set)")
plt.xlim(0, 1)
plt.ylim(0, 1.02)
plt.legend(loc="lower left")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

14.3 Confusion Matrix Metrics Analysis

```
# === FN/FP error analysis: masks & quick summary ===
import numpy as np
from sklearn.metrics import confusion_matrix, classification_report

need = ["y_prob", "y_pred", "y_true"]
for n in need:
    if n not in globals():
        raise RuntimeError(f"Missing '{n}'. Run the prediction cell first (the one that stores y_prob/y_pred/y_true).")

# Ensure 1D arrays
y_true_arr = np.asarray(y_true).ravel().astype(int)
y_pred_arr = np.asarray(y_pred).ravel().astype(int)

# Positive-score (probability) array (works for both sigmoid and softmax)
if 'y_pos' in globals():
    y_pos_arr = np.asarray(y_pos).reshape(-1)
else:
    # Robust fallback
    yp = np.asarray(y_prob)
    if yp.ndim == 2 and yp.shape[1] > 1:
        pos_idx = POS_IDX if 'POS_IDX' in globals() else 1
        y_pos_arr = yp[:, pos_idx]
    else:
        y_pos_arr = yp.reshape(-1)

# Masks
TP_mask = (y_true_arr == 1) & (y_pred_arr == 1)
TN_mask = (y_true_arr == 0) & (y_pred_arr == 0)
FP_mask = (y_true_arr == 0) & (y_pred_arr == 1) # predicted positive but actually negative
FN_mask = (y_true_arr == 1) & (y_pred_arr == 0) # predicted negative but actually positive

# Counts
TP, TN = TP_mask.sum(), TN_mask.sum()
FP, FN = FP_mask.sum(), FN_mask.sum()
total = y_true_arr.size

print("=== Confusion counts ===")
print(f"TP={TP} | TN={TN} | FP={FP} | FN={FN} | total={total}")
```

```

# Full confusion matrix & basic report
labels = [0, 1]
print("\nConfusion matrix (rows=true, cols=pred):\n", confusion_matrix(y_true_arr, y_pred_arr, labels=labels))
print("\nClassification report:\n", classification_report(y_true_arr, y_pred_arr, target_names=(CLASS_NAMES if
'CLASS_NAMES' in globals() else ['Normal','Violence+'])))

# === Top-K hardest FP/FN listings with scores ===
import numpy as np

K = 20 # how many to print

# Optional: attach paths for readability if you have them
# If you have something like test_items = [(path, class_name), ...] keep it here:
paths = None
if 'test_items' in globals():
    try:
        paths = [p for (p, _) in test_items]
    except Exception:
        paths = None

def _print_rows(indices, title, reverse=False):
    if indices.size == 0:
        print(f"\n[{title}] none")
    return
    # sort by score
    idx_sorted = indices[np.argsort(y_pos_arr[indices])]
    if reverse: # high to low
        idx_sorted = idx_sorted[::-1]
    print(f"\n[{title}] showing {min(K, idx_sorted.size)} of {idx_sorted.size}")
    for i in idx_sorted[:K]:
        s = float(y_pos_arr[i])
        t = int(y_true_arr[i]); p = int(y_pred_arr[i])
        tname = CLASS_NAMES[t] if 'CLASS_NAMES' in globals() and 0 <= t < len(CLASS_NAMES) else str(t)
        pname = CLASS_NAMES[p] if 'CLASS_NAMES' in globals() and 0 <= p < len(CLASS_NAMES) else str(p)
        extra = f" | path={paths[i]}" if (paths is not None and i < len(paths)) else ""
        print(f"[{i:5d}] score={s:0.4f} | true={tname:<9s} pred={pname:<9s}{extra}")

# FP: predicted positive, actually negative → sort by high score first (more severe)
_print_rows(np.where(FP_mask)[0], "FP (pred=1, true=0) — high scores first", reverse=True)

```

```

# FN: predicted negative, actually positive → sort by low score first (more severe)
_print_rows(np.where(FN_mask)[0], "FN (pred=0, true=1) — low scores first", reverse=False)

# === Score distributions: TP/TN vs FP/FN ===
import numpy as np
import matplotlib.pyplot as plt

plt.figure(figsize=(9,6))
bins = np.linspace(0, 1, 41)

plt.hist(y_pos_arr[TP_mask], bins=bins, alpha=0.35, label="TP (score of positive)", density=True)
plt.hist(y_pos_arr[FP_mask], bins=bins, alpha=0.35, label="FP (score of positive)", density=True)
plt.hist(y_pos_arr[TN_mask], bins=bins, alpha=0.35, label="TN (score of positive)", density=True)
plt.hist(y_pos_arr[FN_mask], bins=bins, alpha=0.35, label="FN (score of positive)", density=True)

plt.xlabel("Positive-class score")
plt.ylabel("Density")
plt.title("Score distributions by outcome")
plt.legend()
plt.tight_layout()
plt.show()

# === Threshold sweep: FP/FN trade-off table and plot ===
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import precision_score, recall_score

ths = np.linspace(0.0, 1.0, 41)
rows = []
for th in ths:
    pred = (y_pos_arr >= th).astype(int)
    FP_ = ((y_true_arr == 0) & (pred == 1)).sum()
    FN_ = ((y_true_arr == 1) & (pred == 0)).sum()
    TP_ = ((y_true_arr == 1) & (pred == 1)).sum()
    TN_ = ((y_true_arr == 0) & (pred == 0)).sum()
    prec = precision_score(y_true_arr, pred, zero_division=0)
    rec = recall_score(y_true_arr, pred, zero_division=0)
    rows.append((th, TP_, FP_, TN_, FN_, prec, rec))

```

```

print("th\tTP\tFP\tTN\tFN\tprec\trecall")
for th, TP_, FP_, TN_, FN_, prec, rec in rows[::5]: # print every 5th to keep short
print(f"{th:0.2f}\t{TP_}\t{FP_}\t{TN_}\t{FN_}\t{prec:0.3f}\t{rec:0.3f}")

# Plot FP vs FN against threshold
arr = np.array(rows, dtype=float)
plt.figure(figsize=(8,4))
plt.plot(arr[:,0], arr[:,2], label="FP", lw=2) # arr[:,2] is FP_ in our tuple? Careful: rows = (th, TP, FP, TN, FN, ...)
# Correct indices: th=0, TP=1, FP=2, TN=3, FN=4, prec=5, rec=6
plt.plot(arr[:,0], arr[:,4], label="FN", lw=2)
plt.xlabel("Threshold")
plt.ylabel("Count")
plt.title("FP/FN vs Threshold")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```

15. Error Analysis - Misclassified Cases Investigation

```
# === FN/FP error analysis: masks & quick summary ===
import numpy as np
from sklearn.metrics import confusion_matrix, classification_report

need = ["y_prob", "y_pred", "y_true"]
for n in need:
    if n not in globals():
        raise RuntimeError(f"Missing '{n}'. Run the prediction cell first (the one that stores y_prob/y_pred/y_true).")

# Ensure 1D arrays
y_true_arr = np.asarray(y_true).ravel().astype(int)
y_pred_arr = np.asarray(y_pred).ravel().astype(int)

# Positive-score (probability) array (works for both sigmoid and softmax)
if 'y_pos' in globals():
    y_pos_arr = np.asarray(y_pos).reshape(-1)
else:
    # Robust fallback
    yp = np.asarray(y_prob)
    if yp.ndim == 2 and yp.shape[1] > 1:
        pos_idx = POS_IDX if 'POS_IDX' in globals() else 1
        y_pos_arr = yp[:, pos_idx]
    else:
        y_pos_arr = yp.reshape(-1)

# Masks
TP_mask = (y_true_arr == 1) & (y_pred_arr == 1)
TN_mask = (y_true_arr == 0) & (y_pred_arr == 0)
FP_mask = (y_true_arr == 0) & (y_pred_arr == 1) # predicted positive but actually negative
FN_mask = (y_true_arr == 1) & (y_pred_arr == 0) # predicted negative but actually positive

# Counts
TP, TN = TP_mask.sum(), TN_mask.sum()
FP, FN = FP_mask.sum(), FN_mask.sum()
total = y_true_arr.size

print("=== Confusion counts ===")
print(f"TP={TP} | TN={TN} | FP={FP} | FN={FN} | total={total}")
```

```
# Full confusion matrix & basic report
labels = [0, 1]
print("\nConfusion matrix (rows=true, cols=pred):\n", confusion_matrix(y_true_arr, y_pred_arr, labels=labels))
print("\nClassification report:\n", classification_report(y_true_arr, y_pred_arr, target_names=(CLASS_NAMES if
'CLASS_NAMES' in globals() else ['Normal','Violence+'])))
```

16. Final Summary Report

```
# Calculate required metrics for the report
from sklearn.metrics import accuracy_score

# Extract test_loss and test_accuracy from model evaluation
test_results = best_model.evaluate(X_test, y_test_encoded, verbose=0)
test_loss = test_results[0]
test_accuracy = test_results[1] if len(test_results) > 1 else accuracy_score(y_true, y_pred_labels)

# Calculate ROC AUC if not already calculated
if 'roc_auc' not in locals():
    from sklearn.metrics import roc_curve, auc
    fpr, tpr, _ = roc_curve(y_true, y_pred_probs)
    roc_auc = auc(fpr, tpr)

# Find misclassified samples
misclassified_indices = np.where(y_pred_labels.flatten() != y_true)[0]

# Generate comprehensive summary report
summary_report = f"""
{'='*70}
VIOLENCE DETECTION MODEL
FINAL EVALUATION REPORT
{'='*70}

PROJECT DETAILS:
- Course: 31256 Image Processing and Pattern Recognition
- Dataset: Smart-City CCTV Violence Detection (SCVD)
- Model: CNN (MobileNetV2) + LSTM Hybrid
- Task: Binary Classification (Normal vs Violence)

DATASET STATISTICS:
- Training samples: {len(X_train)}
- Test samples: {len(X_test)}
- Image size: {IMG_SIZE}x{IMG_SIZE}
- Frames per video: {FRAME_COUNT}
- Classes: {encoder.classes_}

TRAINING CONFIGURATION:
```

- Epochs trained: `{len(history.history['loss'])}`
- Batch size: `{BATCH_SIZE}`
- Learning rate: `{LEARNING_RATE}`
- Optimizer: Adam
- Loss function: Binary Crossentropy
- Class weights applied: Yes (handling imbalance)

PERFORMANCE METRICS (Test Set):

- Accuracy: `{test_accuracy:.4f}` (`{test_accuracy*100:.2f}%`)
- Loss: `{test_loss:.4f}`
- AUC Score: `{roc_auc:.4f}`
- Misclassified: `{len(misclassified_indices)} / {len(y_test_encoded)}`

PDF REQUIREMENT CHECK:

- Target Accuracy ($\geq 75\%$): `{ '✓ PASSED' if test_accuracy >= 0.75 else 'X FAILED' }`
- Balanced Metrics: ✓ Precision, Recall, F1-Score computed
- ROC/AUC Analysis: ✓ Completed
- Confusion Matrix: ✓ Generated
- Class Weighting: ✓ Applied

IMPROVEMENTS FROM ORIGINAL CODE:

- ✓ Fixed data leakage (sorted file loading)
- ✓ Implemented class weighting
- ✓ Added ROC/AUC evaluation
- ✓ Enhanced error analysis
- ✓ Fixed FRAME_COUNT consistency (10 frames)
- ✓ Added learning rate scheduler

OUTPUT FILES GENERATED:

- Model: `{MODEL_CHECKPOINT_PATH}`
- Training curves: `/Users/hp/UTS/training_history.png`
- Confusion matrix: `/Users/hp/UTS/confusion_matrix.png`
- ROC curve: `/Users/hp/UTS/roc_curve.png`
- Confidence dist: `/Users/hp/UTS/confidence_distribution.png`

`{ '='*70 }`

END OF REPORT

`{ '='*70 }`

```
print(summary_report)

# Save report to file
with open('/Users/hp/UTS/evaluation_report.txt', 'w') as f:
    f.write(summary_report)

print("\n✓ Summary report saved to: /Users/hp/UTS/evaluation_report.txt")
```

A2. Calss Distribution in Train and Test Dataset

Train Dataset:

Normal: 872
Violence: 970
Weaponized: 832
TOTAL: 2674

Test Dataset:

Normal: 169
Violence: 118
Weaponized: 190
TOTAL: 477

A3. Data Loading and Preprocessing

```
Loading training data...
Processing Normal: 650 videos...
  ✓ Loaded 650 valid videos for Normal
Processing Violence: 650 videos...
  ✓ Loaded 649 valid videos for Violence
Processing Weaponized: 650 videos...
  ✓ Loaded 650 valid videos for Weaponized

Training data shape: (1949, 10, 128, 128, 3)
Training labels shape: (1949,)
Unique classes: ['Normal' 'Violence' 'Weaponized']
Class distribution:
  Normal: 650
  Violence: 649
  Weaponized: 650
```

```
Loading test data...
Processing Normal: 169 videos...
  ✓ Loaded 169 valid videos for Normal
Processing Violence: 118 videos...
  ✓ Loaded 118 valid videos for Violence
Processing Weaponized: 190 videos...
  ✓ Loaded 190 valid videos for Weaponized

Test data shape: (477, 10, 128, 128, 3)
Test labels shape: (477,)
Unique classes: ['Normal' 'Violence' 'Weaponized']
Class distribution:
  Normal: 169
  Violence: 118
  Weaponized: 190
```

A4. Model Architecture Summary

Model: "CNN_LSTM_Violence_Detection"

Layer (type)	Output Shape	Param #
time_distributed (TimeDistributed)	(None, 10, 4, 4, 1280)	2,257,984
time_distributed_1 (TimeDistributed)	(None, 10, 1280)	0
time_distributed_2 (TimeDistributed)	(None, 10, 1280)	0
time_distributed_3 (TimeDistributed)	(None, 10, 1280)	5,120
LSTM_1 (LSTM)	(None, 10, 128)	721,408
LSTM_2 (LSTM)	(None, 64)	49,408
FC_1 (Dense)	(None, 256)	16,640
batch_normalization_1 (BatchNormalization)	(None, 256)	1,024
dropout_1 (Dropout)	(None, 256)	0
FC_2 (Dense)	(None, 128)	32,896
batch_normalization_2 (BatchNormalization)	(None, 128)	512
dropout_2 (Dropout)	(None, 128)	0
FC_3 (Dense)	(None, 64)	8,256
dropout_3 (Dropout)	(None, 64)	0
Output (Dense)	(None, 1)	65

Total params: 3,093,313 (11.80 MB)

Trainable params: 832,001 (3.17 MB)

Non-trainable params: 2,261,312 (8.63 MB)

9. Training Callbacks Setup

EarlyStopping, ModelCheckpoint, and ReduceLRonPlateau

```
# Setup training callbacks
callbacks = [
    # Early stopping to prevent overfitting
    EarlyStopping(
        monitor='val_loss',      # track validation loss
        patience=PATIENCE,      # reduce LR by half when triggered
        restore_best_weights=True,
        verbose=1                # print log messages when LR changes
    ),

    # Save best model
    ModelCheckpoint(
        MODEL_CHECKPOINT_PATH,
        monitor='val_loss',
        save_best_only=True,
        mode='min',
        verbose=1
    ),

    # Reduce learning rate on plateau
    ReduceLRonPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=2,
        min_lr=1e-7,
        verbose=1
    )
]

print("Callbacks configured:")
print(" - EarlyStopping (patience=10)")
print(" - ModelCheckpoint (save best model)")
print(" - ReduceLRonPlateau (adaptive learning rate)")
```

2] ✓ 0.0s

```
Callbacks configured:
- EarlyStopping (patience=10)
- ModelCheckpoint (save best model)
- ReduceLRonPlateau (adaptive learning rate)
```